

Intelligent Dead Reckoning (IDR) System — Project Documentation

SIH Problem Statement #26168 — ISRO / Department of Space — Theme: Smart Vehicles

1. Project Title & Context

Intelligent Dead Reckoning (IDR) with GNSS Fusion — a smartphone-based and edge-deployable navigation system that maintains accurate vehicle position tracking through GNSS outages (tunnels, underground parking, urban canyons, jamming), built for Smart India Hackathon problem statement #26168, sponsored by ISRO under the Smart Vehicles theme.

2. Problem Statement Summary

Most navigation apps depend entirely on GNSS (GPS/Galileo/NavIC). When GNSS signal disappears — inside tunnels, underground parking, dense urban canyons, or during jamming — the map freezes, the vehicle marker jumps, or the position becomes wrong. This is a serious problem for logistics, ride-hailing, quick commerce, and emergency responders, and it disproportionately affects the vast majority of Indian vehicles (trucks, older cars, two-wheelers) that have no factory-fitted INS and rely solely on a dashboard-mounted or loosely-held smartphone.

The task is to build an AI/ML-enhanced dead-reckoning and sensor-fusion system that uses only the smartphone's built-in sensors (accelerometer, gyroscope, magnetometer, GNSS chip) — no OBD-II/CAN-bus connection — to estimate position accurately during GNSS blackout, and to switch seamlessly back to GNSS-aided tracking on recovery.

Hard performance targets:

Positional drift < 10% of distance traveled during GNSS blackout (e.g., <5m over 50m in <1 min; <100m over 1km at 60km/h).

Position update rate: ~10Hz on smartphone; ~200Hz on an edge-deployable engine using FOG-grade IMU input.

3. Core Concepts

Concept	Plain-Language Definition
GNSS	Satellite-based positioning (GPS, Galileo, NavIC); accurate but blocked by tunnels, buildings, jamming
IMU	Accelerometer (linear acceleration) + Gyroscope (rotation rate) + Magnetometer (compass heading) — built into every phone
Dead Reckoning	Estimating current position from last known position + speed + direction + elapsed time, without external positioning
Drift	The exponentially compounding position error that results from integrating small sensor errors over time
Sensor Fusion	Intelligently blending GNSS (when available) with IMU-derived motion to produce one continuous, accurate estimate
Kalman Filter (EKF)	The mathematical mechanism for that blending — constantly re-weighting trust between GNSS and INS based on current reliability
Map Matching	Snapping the estimated position onto the real road network to correct drift using the fact that vehicles travel on roads
Non-Holonomic Constraints (NHC)	Physical rules (a car doesn't move sideways or vertically) used as extra correction signals

4. Solution Overview

The system uses GNSS whenever available, and switches to AI-enhanced dead reckoning the instant it isn't — without a visible jump on the map, and without needing any vehicle hardware connection. Six core capabilities compose the full solution:

In-Vehicle Alignment & Calibration Engine — determines phone orientation (pitch/roll/yaw) relative to the vehicle, regardless of mount type, with automatic re-calibration if the phone moves.

AI Speed & Vibration Filter — a CNN+LSTM/GRU model that filters road noise/potholes and estimates true forward velocity directly from IMU signals, with no speedometer/OBD dependency.

Map-Matching & Kinematic Constraints — Hidden Markov Model map matching against OpenStreetMap, combined with non-holonomic constraints, to bind the estimated trajectory to real roads.

GNSS+INS Fusion Engine — an Extended Kalman Filter (position, velocity, orientation, sensor biases) enhanced with an AI-learned correction/adaptive-noise component.

Seamless GNSS Deficit Handler — continuous, gradual transition logic between GNSS-Available, Degraded, and Denied states, built into the fusion filter's natural behavior rather than a hard switch.

Real-Time Navigation Interface — a polished mobile application with a smoothly animated, uninterrupted vehicle icon and clear visual communication of system state.

5. System Architecture

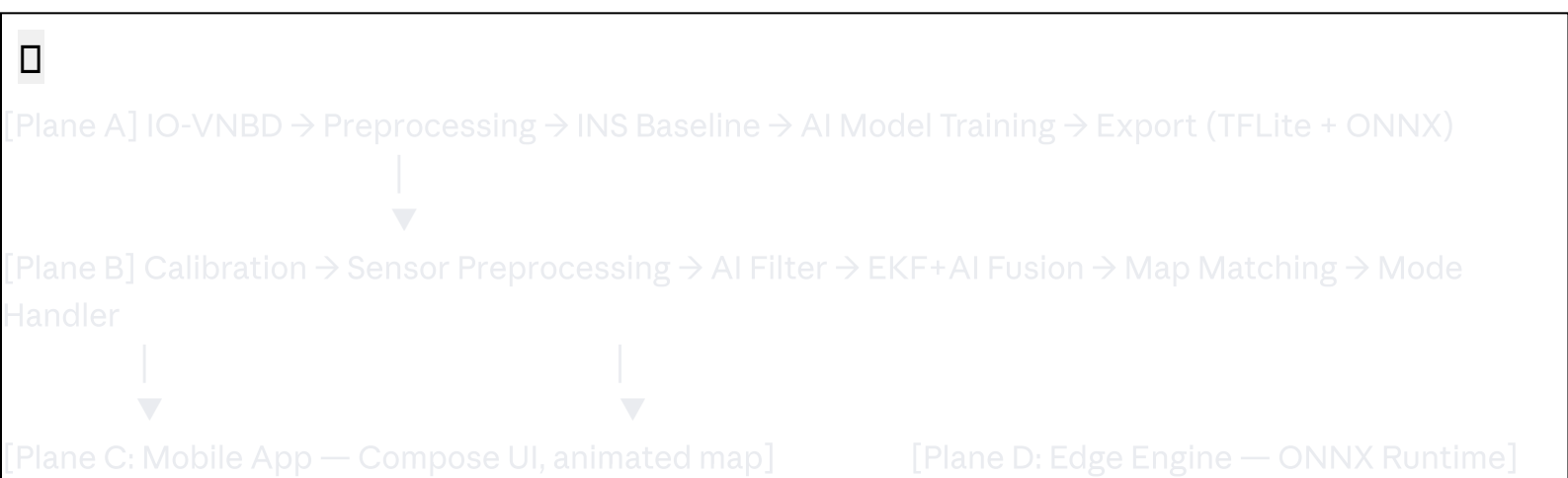
Four architectural planes:

Plane A — Offline Training Pipeline (Cloud/Desktop): dataset processing, INS mechanization baseline, AI model training, evaluation, export.

Plane B — Shared On-Device Inference Core: calibration, sensor preprocessing, fusion, map matching, mode-switching — identical algorithmic logic used by both deployment targets.

Plane C — Mobile Application (Android, Kotlin, Jetpack Compose): sensors, UI, TFLite inference, map rendering, animation.

Plane D — Edge-Deployable Engine (Python/ONNX Runtime): higher-rate FOG IMU ingestion, same fusion core, packaged for edge hardware.



6. Dataset — IO-VNBD

Source: github.com/onyekpeu/IO-VNBD (Onyekpe et al., *Data in Brief*, 2020), CC BY licensed.

Contents: synchronized smartphone IMU (10Hz) + smartphone GPS + vehicle sensors + Racelogic VBOX ground truth (high-precision reference).

Scale: ~40 hours/1,300km vehicle data; ~58 hours/4,400km smartphone data; collected in UK, Nigeria, France across varied road/driving scenarios.

Documented noise characteristics: ~0.15g acceleration noise, ~0.08 rad/s yaw-rate noise under vibration/braking.

Published reference results: neural-network correction achieving up to 89.55% improvement in INS displacement estimation over raw INS — the benchmark class this project targets.

7. AI/ML Approach

Speed & Vibration Filter: 1D CNN (local vibration/noise pattern extraction) → LSTM/GRU (temporal integration) → dense output (forward velocity prediction), trained on windowed IMU data with VBOX ground truth as the label.

Fusion Enhancement: an AI-learned adaptive noise/residual-correction layer on top of the classical EKF, replacing fixed hand-tuned covariances with learned, context-sensitive adjustment.

Deployment: trained on cloud/desktop (TensorFlow/Keras), exported to TensorFlow Lite (mobile) and ONNX (edge), quantized for real-time on-device inference.

8. GNSS+INS Fusion & Outage Handling

Extended Kalman Filter state: position, velocity, orientation (quaternion), accelerometer bias, gyroscope bias. GNSS fixes serve as measurement updates when available; between fixes, the filter propagates using AI-corrected INS mechanization. GNSS quality is classified as Available / Degraded / Denied via HDOP/satellite-count thresholds. There is no separate "switch" — the filter naturally continues on INS-only when GNSS updates stop, and re-converges smoothly (not a hard snap) when GNSS returns.

9. Map Matching

OpenStreetMap road network extracted offline via OSMnx. Hidden Markov Model (Viterbi decoding) snaps the fused position estimate onto the most plausible road segment. Non-holonomic constraints (no lateral/vertical vehicle motion in vehicle frame) are applied as pseudo-measurements, tightening the filter especially during GNSS-denied stretches.

10. Mobile Application — UI/UX Vision

The app is built to a genuinely polished standard, not a functional-only prototype:

Framework: Jetpack Compose (not legacy Android Views) — chosen specifically because its animation APIs match the app's core need: smoothly reacting to a continuous stream of position/state updates.

Smooth marker movement: eased interpolation between each 10Hz position update, so the vehicle glides rather than ticks.

Camera behavior: smooth pan/rotate to follow vehicle heading, matching real navigation app conventions.

Live state visualization: a signal/status indicator that visibly and smoothly transitions between GNSS-Available/Degraded/Denied states, directly communicating real system state (not decorative).

Animated confidence/drift indicator: an accuracy circle around the vehicle icon that grows during GNSS-Denied periods and shrinks on recovery — makes the underlying drift math visible to the user.

Guided calibration flow: an onscreen sequence ("mount phone, drive straight briefly") with progress animation, making the calibration step legible rather than invisible.

Smooth screen transitions, Material 3 theming, dark/day-night map modes, haptic feedback on mode switches.

11. Edge-Deployable Engine

Python-based, using ONNX Runtime for model inference, sharing the same calibration/fusion/map-matching core logic as the mobile app (not a reimplementation). Designed to ingest higher-rate (~200Hz) FOG IMU data, rate-agnostic in its internal loop structure so the same algorithms scale from 10Hz to 200Hz without redesign.

12. Technology Stack

Layer	Technology
Training	Python, TensorFlow/Keras, pandas, numpy, scipy
Classical filtering	<code>filterpy</code> or custom NumPy EKF/UKF
Map matching	OSMnx + custom HMM/Viterbi
Mobile platform	Android, Kotlin, Jetpack Compose
Mobile inference	TensorFlow Lite
Mobile maps	Google Maps SDK or <code>osmdroid</code> (offline OSM)
Edge engine	Python, ONNX Runtime
Version control	Git

13. Evaluation & Benchmark Strategy

Primary metric: positional drift (meters, and % of distance traveled) during simulated/real GNSS-denied stretches, matched against the competition's exact benchmark scenarios (50m/<1min, 1km/60kmph).

Secondary metrics: velocity RMSE/MAE, orientation RMSE, all against VBOX ground truth. **Method:** mask GNSS in software over defined stretches of held-out IO-VNBD sequences (and later, real field-test drives), measure final position error against known ground truth on GNSS reacquisition.

14. Repository Structure

```
└─ dr-system/
   └─ data/{raw,processed}/
   └─ training/ (preprocessing, ins_mechanization, model training, export)
   └─ core/ (calibration, ekf_fusion, map_matching, mode_handler — shared)
   └─ mobile/ (Android project, Compose UI)
   └─ edge/ (Python edge engine)
   └─ models/ (.tflite, .onnx artifacts)
   └─ reports/ (schema docs, evaluation plots, proposal)
   └─ tests/
   └─ README.md
```

15. Security & Data Handling

No cloud dependency for inference (architectural requirement and privacy benefit). No transmission of raw GNSS/IMU data off-device without explicit purpose. Offline map data stored locally, avoiding live location leakage to third-party tile servers during navigation.

16. Deliverables

Screening-stage evidence (produced as a natural byproduct of early AI/data-science work): dataset schema documentation, raw-INS baseline plot, AI-corrected comparison plot, short written proposal.

Finale-stage deliverable: fully working, polished mobile application; edge-deployable engine; both benchmarked against the drift and update-rate targets using additional datasets provided at that stage.